Technical Documentation

CashPilot: an AI financial assistant for small and medium businesses · Oct 20, 2026 · @Arnob
This document describes what is built and running. Items not built yet are marked **planned**.

Introduction

CashPilot is an AI financial assistant for small and medium businesses. It warns owners about cash shortages 7 to 30 days in advance, tells them who to collect money from first, builds an action plan to close the gap, gives them an explainable credit score, and estimates how much they can safely borrow. Live: **https://cashpilot-ai-demo.vercel.app** · Source: **https://github.com/GaziArnob/CashpPlot** (branch `cashpilot-final`).

1. System Architecture

The system has four layers: a Next.js web client, a FastAPI application server, an intelligence layer of deterministic engines plus an AI agent, and a PostgreSQL data layer. Requests flow top-down; the agent reaches data only by calling the engines as tools.

#	Feature	What it does
1	Cash Flow Prediction	Projects the daily balance 7 to 30 days ahead, flags shortage days, and runs what-if scenarios
2	Cash Advisor	Turns a shortage into an ordered action plan (collect, clear stock, delay a bill, cut spending), each step simulated on the forecast
3	Smart Payment Recovery	Ranks customers by collection priority, suggests an action and a credit limit, and drafts reminders (WhatsApp or email, sent only after the owner approves)
4	Supplier Payment Plans	Detects payment clashes, proposes instalments, compares financing and drafts the supplier message
5	AI Credit Scoring	Explainable 300 to 900 score from income, cash flow, dues and repayment history, plus a "how to raise it" planner
6	Loan Readiness & Recommendation	Safe loan amount and matching products from an 8-type loan catalog (an estimate, not an approval); consent-based lender introductions
7	Product Performance	Margin, trend and stock-days per product; push, keep, shrink or clear
8	Notifications	16 alert rules, unread count, preferences, optional urgent email
9	AI Financial Agent	Answers questions in plain language using seven read-only engine tools

1.1 Design principle: deterministic core, AI interface

Every financial number is computed by deterministic, testable engines. The LLM never calculates figures; it interprets the user's intent, picks which engine to call, and explains the result. Any number in an AI answer that the engines did not produce is detected and the answer is replaced by a template. This keeps every output auditable and prevents hallucinated numbers.

No paid AI services. All AI features run on free options behind one LLM provider layer; the provider order is one setting (`LLM_PROVIDERS=gemini,groq,ollama`). If a free-tier rate limit is hit or a provider fails, the request falls back to the next one, and finally to templates, so the product works with no AI key at all.

1.2 Components

Order	Provider	Cost	Used for
1	Google Gemini API (free tier)	Free, rate-limited	Agent chat with tool calling, explanations, receipt reading (vision fallback)
2	Groq API (free tier, open-source models)	Free, rate-limited	Fallback for chat and text generation
3	Ollama with an open model on the server (e.g. Qwen 2.5 7B)	Free, offline	Local-only mode; off unless enabled (`OLLAMA_ENABLED`)
–	Tesseract OCR	Free, open source	Receipt text extraction without any API; Gemini vision is used only if Tesseract is not installed
–	Templates	Free	Reminder and explanation text if every LLM provider is unavailable

Component	Layer	Responsibility
Web client	Presentation	Pages, charts, chat interface, form validation (Next.js 16, React 19)
API routes	Application	Request validation, authentication, plan checks, rate limiting (`app/api/`)
Forecast Engine	Intelligence	Projects daily cash 7 to 30 days ahead and flags shortages
Collection (Recovery) Engine	Intelligence	Ranks overdue receivables, suggests actions and credit limits
Credit Engine and Credit Plan	Intelligence	Explainable score from 300 to 900, and the effect of each improving action
Loan Engine and Loan Catalog	Intelligence	Safe borrowing capacity, 8 loan types matched by score and amount
What-if Engine	Intelligence	Re-runs the forecast under user-defined scenarios
Planner (Cash Advisor)	Intelligence	Ordered action plan, each step simulated against the forecast
Payables Engines	Intelligence	Dated cash flow, payment clashes, instalment plans, financing comparison
Product Engine	Intelligence	Margin, turnover and cash conversion per product
Insight Engine	Intelligence	Spending anomalies and period-over-period changes; P&L reports
Alert rules and Daily job	Application	16 isolated rules create notifications; a daily job refreshes scores, plans and sends urgent email
Messaging layer	Application	Owner-approved delivery: email (Mailjet, Brevo or SMTP) and WhatsApp (Bird, Meta Cloud API or Twilio)
AI Agent	Intelligence	Natural-language interface that calls the engines as tools
PostgreSQL	Data	Persistent data isolated per business (SQLite for local development)

1.3 Frontend pages

#	Page	Route	Contains
1	Landing	/	Product intro and call to action
2	Auth and Onboarding	/login, /signup, /onboarding	Login, signup, a three-step business setup wizard

3	Dashboard	/dashboard	KPI cards, cash alerts, daily AI summary, collect-first list
4	Cash Advisor	/advisor	Shortage plan with approve, done and dismiss
5	Notifications	/notifications	Alert list, unread/all tabs, mark read
6	Transactions	/transactions	List, add, CSV import, receipt scan, bank import (Plaid sandbox)
7	Cash Flow	/cashflow	Forecast chart, shortage days, calendar, what-if
8	Ledger	/ledger	Receivables (invoices, recovery, reminders) and payables (suppliers, bills)
9	Customers	/customers	Customer list, profile, payment history, risk level, edit
10	Supplier Plans	/payables	Payment clashes, instalment plans, financing comparison, drafts
11	Products	/products	Product performance quadrant, sales and stock
12	Credit Score	/credit-score	Score gauge, factors, history, "how to raise your score"
13	Loan Center	/loan	Readiness, recommended amount, 8 loan types, checklist, what-if, My loans, introduction requests
14	Reports	/reports	P&L, expense insights, goals, PDF export
15	AI Agent	/agent	Chat, suggested questions
16	Settings	/settings	Business profile, cash threshold, reminder tone, AI privacy, plan, team, two-factor, lender access, notifications, data export
17	Lender view	/lender	Read-only, consent-scoped view for a lender with a token

1.4 Data model

Every business-owned table carries a `business_id`, which is how tenant data is isolated. 34 tables, created by eight Alembic migrations.

Group	Tables and key fields
Accounts	`users` (name, email, phone, password_hash, role, TOTP secret, failed_logins, locked_until), `businesses` (name, type, currency, started_at, cash_threshold, plan, ai_mode), `refresh_tokens` (jti, rotation), `audit_log`
Records	`transactions` (date, amount, type income/expense, category, source manual/csv/scan/bank), `customers`, `invoices`, `invoice_payments`, `suppliers`, `bills`, `recurring_expenses`, `products`, `sale_items`, `stock_movements`
Engines and plans	`collection_scores`, `product_scores`, `credit_scores` (score, factors JSON), `recommendations`, `action_logs`, `payables_recommendations`, `payment_plans`, `payment_plan_installments`, `negotiation_drafts`, `financing_offers`
Credit and loans	`loans`, `repayments`, `loan_products` (the 8-type catalog), `lender_referrals`, `lender_access` (scope, expires_at, revoked)
Communication	`reminders` (message, tone, channel, status), `notifications` (type, severity, dedupe key, link, is_read), `chat_messages`, `goals`
Integrations	`bank_connections` (the bank access token is stored encrypted and never sent to the browser)

1.5 Main API endpoints

Module	Endpoints
Auth	`POST /auth/signup, /auth/login, /auth/refresh, /auth/logout`; `GET /auth/me`; `POST /onboarding`; `POST /auth/2fa/setup, /auth/2fa/enable, /auth/2fa/disable`
Dashboard	`GET /dashboard/summary`
Transactions	`GET, POST /transactions`; `POST /transactions/import`; `POST /transactions/scan`
Cash flow	`GET /cashflow/forecast?days=30`; `GET /cashflow/calendar`; `GET /cashflow/upcoming`
Cash Advisor	`GET /action-plan`; `POST /actions/{id}/mark-done, /dismiss, /reopen`; `GET /cash-advisor/summary`
Ledger	`GET, POST /invoices`; `PATCH /invoices/{id}/paid`; `POST /invoices/{id}/payments`; `GET /recovery/priority`; `POST /recovery/reminder/{invoice_id}`; `POST /reminders/{id}/send`; `GET /messaging/channels`; `GET, POST /suppliers, /bills`
Customers	`GET, POST /customers`; `PUT, GET /customers/{id}`; `POST /customers/{id}/credit-limit`; `GET /collections/priority`
Supplier plans	`GET /payables/analysis`; `POST, GET /payables/recommendations`; `GET /financing/compare`; `/payment-plans` (create, update, approve, supplier-response, cancel); `/negotiation-drafts`
Products	`GET, POST /products`; `GET /products/performance`; `POST /sale-items, /stock-movements`
Credit	`GET /credit-score`; `GET /credit-score/plan`; `GET /credit-score/history`
Loan	`GET /loan/readiness, /loan/recommendations`; `POST /loan/what-if`; `/loans` (list, create, delete, repayments); `/referrals`

Reports	`GET /reports/pnl, /reports/insights, /reports/export`; `/goals`
Notifications	`GET /notifications`; `/notifications/summary`; `POST /notifications/read-all`; `PATCH /notifications/{id}/read`; `DELETE`; `GET, PUT /notifications/preferences`
Agent	`POST /agent/chat` and `/agent/chat/stream` (streaming); `GET /agent/history, /agent/suggestions`
Settings and team	`GET, PUT /settings`; `GET, POST, DELETE /lender-access`; `GET /lender/view`; `GET /settings/export`; `DELETE /settings/account`; `/team` (add, remove, reactivate, reset password)
Billing and bank	`GET, POST /billing/plan` (demo mode); Stripe checkout and webhook (off by default); `/bank` (status, link, exchange, sync, delete) with Plaid sandbox

1.6 Request flow example

A user asks: "Will I run short on cash next week?"

1. The client sends `POST /agent/chat`.
2. The Agent Service sends the question and the tool schemas to the LLM.
3. The LLM requests `get_cash_forecast(days=7)`.
4. The Agent Service calls the Forecast Engine with the `business_id` from the session; the engine reads transactions, invoices and bills.
5. The engine returns a shortage date and gap amount.
6. The Agent Service passes the result back to the LLM.
7. The LLM writes the answer, for example: "You may be short about 12,000 on Oct 22. Rahim Store owes you 15,000, 10 days overdue; collecting it covers the gap."
8. Any figure in the answer is checked against the tool results, and the answer streams back to the client.

1.7 Technology stack

Layer	Technology
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4, Recharts
Backend	Python, FastAPI, Pydantic 2, SQLAlchemy 2, Alembic (migrations)
Analytics	NumPy; an own Holt-Winters implementation (additive, damped trend); scikit-learn (logistic regression for payment probability)
AI	Free-tier LLM APIs: Google Gemini (primary), Groq (fallback); Ollama local model as an optional offline mode; Tesseract OCR for receipts
Database	PostgreSQL with row-level security (Neon); SQLite for local development
Integrations	Mailjet, Brevo or SMTP (email); Bird, Meta Cloud API or Twilio (WhatsApp); Plaid sandbox (bank import); Stripe (code present, off by default)
Deployment	Vercel (frontend), Render (backend, Docker), Neon (database). All free tiers.

2. Core Algorithm Description

Several engines do the analysis. Each is deterministic and testable, and the AI agent only calls them as tools.

2.1 Cash flow prediction

Goal: project the daily cash balance for the next H days (H = 7 to 30) and flag shortages.

1. **Daily series.** Group transactions by day into income I(t) and expense E(t), over the last 180 days. Scheduled items (rent, salaries, loan installments, bills) are removed from the trend model and placed on their exact day instead, because a few large monthly payments distort a daily trend. Outliers are capped.
2. **Baseline forecast.** Holt-Winters additive smoothing with a damped trend and weekly seasonality (period 7). With less than 60 days of history, fall back to a 28-day weighted moving average multiplied by a day-of-week factor (average of each weekday over the last 12 weeks).
3. **Known future events.** Add expected invoice inflows, weighted by each customer's historical on-time payment rate, and subtract bills, loan installments and recurring payables on their due dates. Each amount is counted once and tagged *confirmed* (known obligations) or *estimated* (forecast sales and running costs).
4. **Balance projection.** Roll the balance forward day by day.
5. **Shortage detection.** Flag every day where the projected balance falls below the business's cash threshold.

$$\hat{I}(t) = \mathrm{WMA}_{28}(I) \cdot \mathrm{DOW_I}(t),\ \hat{E}(t) = \mathrm{WMA}_{28}(E) \cdot \mathrm{DOW_E}(t)$$
$$B(t) = B(t-1) + \hat{I}(t) - \hat{E}(t) + \Sigma_{\text{invoices due } t}\, a_i \cdot \mathrm{p_collect}(c_i) - \Sigma_{\text{bills due } t}\, b_j$$
$$\text{Shortage at } t \Leftrightarrow B(t) < \text{threshold},\ \mathrm{Gap}(t) = \text{threshold} - B(t)$$

An 80% confidence band (±1.28 times the standard deviation of past one-step forecast errors) is drawn around the line. A day is marked "at risk" when only the lower edge of the band crosses the threshold. **Output:** daily projected balance, shortage dates, gap amounts and the main drivers of each shortage.

2.2 Smart payment recovery

Goal: rank customers by who should be contacted first, suggest what to do, set a credit limit, and draft a reminder.

$$\mathrm{Priority} = 100 \cdot (0.30 \cdot A + 0.25 \cdot D + 0.15 \cdot R + 0.15 \cdot C + 0.15 \cdot P)$$

Feature	Definition (normalized 0 to 1)	Weight
A	Amount owed, log-scaled against the largest balance	0.30
D	Aging bucket weight: current 0.05, 0-30 days 0.35, 31-60 days 0.60, 61-90 days 0.85, 90+ days 1.00	0.25
R	Payment-history risk = 1 − on-time rate (new customers 0.5)	0.15
C	How much of the forecast cash gap the overdue amount covers	0.15
P	Probability of a payment within 7 days: a logistic regression trained on this business's own past invoices when there are at least 40 examples and 10 positives, otherwise transparent rules	0.15

The score maps to a suggested action (stop credit and visit, visit, call, offer a 2% early-payment discount, send a reminder, or wait) and to a reminder tone (friendly, firm or final) that follows how late the invoice is and how risky the customer is. A new customer gets a starting credit limit. The LLM fills a template with only the customer's name, amount, due date and tone; it sees no other customer data, and its output is checked against the template's figures.

2.3 AI credit scoring (explainable)

Goal: a score from 300 to 900 with transparent reasons, computed over the last 6 to 12 months.

Factor	Definition (normalized 0 to 1)	Weight	Healthy baseline
f1	Average monthly income, log-scaled (about 500 to 500,000)	0.20	0.50
f2	Income stability = 1 − coefficient of variation	0.15	0.60
f3	Share of months with positive net cash flow	0.20	0.75
f4	1 − (outstanding receivables ÷ monthly income), clipped to 0 to 1	0.15	0.70
f5	On-time repayment rate (neutral 0.5 when there is no repayment history yet)	0.20	0.90
f6	Business age in months ÷ 60, capped at 1	0.10	0.40

$$S = 300 + 600 \cdot \Sigma_{i=1}^{6} w_i f_i \quad \text{Contribution}_i = 600 \cdot w_i \cdot (f_i - b_i)$$

Score	Band
750 to 900	Excellent
650 to 749	Good
550 to 649	Fair
300 to 549	Needs improvement

Explainability. Each factor's contribution is measured against a baseline b_i, the value for a typical healthy business. The largest positive and negative contributions are shown as reasons, for example: "High outstanding receivables lowered your score by 38 points." A new account with little data is labelled "only a starting point" and told exactly which data to add, and the confidence of the score is shown.

How to raise it. The planner re-runs the same formula for each possible action (for example collecting a specific overdue invoice, or paying the next bills on time) and reports the points each would earn and the steps to the next band and the next loan type. Nothing is guessed.

Planned: once real repayment outcomes are available, a gradient-boosted model could replace the fixed weights, with SHAP values keeping each score explainable.

2.4 Loan readiness and recommendation

Goal: estimate a safe loan amount and matching products. The output is an estimate, not a loan approval.

1. **Conservative capacity.** Take the 25th percentile of monthly net cash flow over the last 12 months (NCF_{p25}), so one strong month cannot inflate capacity.
2. **Safe monthly installment.** Take a share α of that, set by credit band, minus existing installments: α = 0.40 (Excellent), 0.35 (Good), 0.25 (Fair), 0.15 (Needs improvement).
3. **Maximum principal.** Invert the standard EMI formula, with r the monthly interest rate and n the tenure in months.
4. **Product matching.** The loan catalog has 8 loan types with minimum scores from 500 to 700 and different rates. Keep the products where the business's score meets `min_score` and P_{max} covers `min_amount`; rank by total cost of credit, lowest first. Each type is shown with a status such as available, too small to repay, or score too low.
5. **Readiness checklist.** Rule-based actions that would raise the score or capacity, such as "Collect receivables older than 30 days".

$$\text{SafeEMI} = \alpha \cdot \text{NCF}_{p25} - \text{ExistingEMI}$$
$$P_{max} = \text{SafeEMI} \cdot ((1+r)^n - 1) \,/\, (r\,(1+r)^n)$$

Owners can record loans they already have (*My loans*); their installments then count in the cash forecast and in the credit score. Introduction requests to a lender need the owner's consent and are only recorded and tracked; nothing is sent to a lender in this version.

2.5 What-if simulator

The user changes inputs, such as a new loan, a sales change of x%, or a new fixed expense, and the Forecast Engine re-runs with them. The original and scenario balance curves are shown side by side.

$$\hat{I}'(t) = \hat{I}(t)(1 + x),\ \hat{E}'(t) = \hat{E}(t) + \text{NewExpense} + \text{NewEMI}$$

2.6 Insight engine

Rule-based checks compare each expense category's last 30 days with its monthly average over the 90 days before. A change over 20% raises an insight, which the LLM phrases for the dashboard summary. The same engine produces the monthly profit-and-loss report.

2.7 Cash Advisor (action plan)

When the forecast shows a shortage, the planner orders the cheapest and safest steps first: (1) collect overdue money from the highest-scoring customers, (2) clear slow and dead stock at a discount, (3) ask suppliers to delay bills that fall before the gap, (4) cut owner drawings and spending that jumped, (5) scale up the star products once there is spare cash, and (6) financing, only for what is still missing after steps 1 to 4. Each step is applied to the forecast to measure its real effect, and the plan shows how much of the gap it covers. The owner marks steps done, dismisses or reopens them; nothing executes by itself.

2.8 Supplier payment plans

The payables engine builds a dated 30-day cash flow in which every amount is counted exactly once and tagged confirmed or estimated, finds days where payments clash with low cash, and proposes steps (instalments, moving a bill, short-term financing) that actually lift the lowest balance above the safety level. A draft message to the supplier is written from a template with the supplier's name masked; the owner approves the plan and records the supplier's reply. Financing offers shown are demo data.

2.9 Product performance

For each product over the last 30 and 90 days: units, revenue and growth, gross margin, profit contribution, inventory turnover, days of stock left, and cash conversion speed (cash versus credit sales and how long credit takes to come back). Products are classed as star, cash cow, slow mover, dead stock or new, and each gets a recommendation (push, keep, shrink, clear).

2.10 AI financial agent

The agent runs a bounded tool-use loop of at most 5 steps. Its tools are `get_cash_forecast`, `get_overdue_customers`, `get_credit_score`, `get_loan_recommendation`, `run_what_if`, `get_spending_insights` and `draft_reminder`.

1. Send the system prompt, the user's question and the tool schemas to the LLM.
2. If the LLM requests a tool, validate the arguments, run the engine with the `business_id` from the authenticated session, and return the result to the LLM.
3. Repeat until the LLM answers in text, or stop at 5 steps with a fallback message.

Three grounding rules apply: every number in an answer must come from a tool result; the model can never choose the `business_id`; and all tools are read-only, so any action such as sending a reminder needs the user's confirmation in the UI. If no LLM provider is reachable, the agent falls back to keyword intents and template answers, so the chat always works.

2.11 Alerts and the daily job

Sixteen rules (cash shortage, cash already low, overdue customers, late and due invoices, bills due, supplier plan payments, loan installments, low stock, credit score drops, spending spikes, sales drops, lender follow-ups, goal deadlines, bank reminders, plan and trial reminders) create notifications with a severity and a link to the page that fixes the problem. Each rule runs in isolation, so one failure does not stop the others, and a dedupe key plus cooldown prevents repeats. A daily job refreshes score snapshots and action plans and can email urgent alerts once per day.

3. Security Design

Every request is authenticated, every query is scoped to one business, contact details are encrypted at field level, and the AI agent can only read data through scoped, read-only tools.

3.1 Authentication

Control	Implementation
Password storage	bcrypt, cost factor 12
Access token	JWT (HS256), 15-minute lifetime
Refresh token	Rotating, 7-day lifetime, stored server-side by id so it can be revoked, in an httpOnly, secure, SameSite=Strict cookie
Brute-force protection	Rate limits per client address and signed-in user (10 auth requests per minute); the account locks for 15 minutes after 5 failed logins
Second factor	Optional authenticator-app code (TOTP, 6 digits, 30 seconds). SMS codes are not used.

3.2 Authorization and tenant isolation

- **business_id scoping.** Every business-owned record has a `business_id`. Every query filters by the business of the authenticated user, never by one sent in the request.
- **Row-Level Security.** PostgreSQL RLS policies (forced, with the business set per request) enforce the same rule in the database as a second line of defence. The database role must not be a superuser. Seed, background jobs and the lender view run with an explicit bypass flag.
- **Object-level checks.** Any ID in a URL, such as `/invoices/{id}`, is checked for ownership before it is read or changed.
- **Roles.** owner (full access), staff (transactions, ledger and customers only), lender (read-only, consent-scoped). Premium features are additionally gated by plan.

3.3 Data protection

Data	Protection
In transit	TLS on every connection (provided by the hosts), HSTS header
At rest	Database and disk encryption provided by the database host
Contact details	Customer phone numbers and emails encrypted at field level (Fernet, AES-based)
Bank access tokens	Stored encrypted and never sent to the browser
Receipts and imports	Processed in memory and not stored; only the confirmed transaction is saved
Backups	Handled by the database host
Secrets	Free-tier LLM API keys, database credentials and provider keys kept in backend environment secrets, never sent to the client or committed to git

3.4 Consent-based lender access

Lenders see a business's score and reports only after the owner grants access. Each grant names its scope (which data), has an expiry date and can be revoked at any time. The token is shown once and stored hashed.

Every lender view is written to the audit log.

3.5 AI-specific security

Free-tier providers may keep or review submitted data, so only minimal fields are sent to them: amounts, dates and category names, with customer and supplier names replaced by codes, and never phone numbers, emails or account details. Owners choose cloud, local-only or no AI in Settings; in local-only mode only a model on the server is used.

Risk	Mitigation
Prompt injection through user data (a transaction note or receipt text)	Tool results and receipt text are passed as data, and the prompts tell the model to ignore any instructions inside them
Cross-tenant data leak	Tools take `business_id` from the server session; the model cannot choose which business to query
Unauthorized actions	All agent tools are read-only; side effects such as sending a reminder need explicit user confirmation
Hallucinated figures	Numbers come only from engine outputs, and any figure in an answer that is not in the tool results causes the answer to be replaced by a template
Over-sharing with the LLM	Only the fields a tool needs are sent; no passwords, tokens or contact lists
Runaway loops and cost abuse	At most 5 tool steps per request, a request timeout, and per-user AI rate limits (20 per minute)

3.6 Application security

- **Input validation:** Pydantic schemas on every endpoint, with type, range and length checks.
- **SQL injection:** ORM with parameterized queries only.
- **Cross-site scripting:** React output escaping; the API replies with `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'`.
- **CSRF:** the refresh cookie is SameSite=Strict, and the cookie-authenticated route also requires a custom `X-Requested-With` header that a cross-site form cannot set.
- **File uploads:** type and size limits (CSV up to 2 MB; JPEG, PNG or WebP images up to 5 MB); files are processed in memory.
- **Rate limiting:** per client address and signed-in user on auth (10 per minute), AI (20 per minute) and upload (10 per minute) endpoints. The limiter is in memory; it would move to Redis for several workers.
- **Security headers:** HSTS, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Cache-Control: no-store`.
- **Messaging:** reminders are sent only when the owner presses Send; subject lines are protected against header injection; a failed send keeps the draft and shows the provider's reason.

3.7 Monitoring and audit

The audit log records signups, logins (including failed ones), onboarding, data exports, lender access grants, views and revocations, reminders sent, payment plan approvals, bank connections, staff changes, two-factor changes, plan changes and settings changes. Application logs never contain passwords, tokens or full financial records. Planned: alerts on logins from unusual locations and on bulk data access.

3.8 Responsible AI and compliance

- Every credit score shows the factors behind it.
- Loan output is labelled "Estimate only, not a loan approval."

- Introduction to a lender needs an explicit consent tick.
- Users can export or delete their data at any time (account deletion also cancels a subscription first).
- The design follows data-minimization and consent principles in line with GDPR-style practice and applicable local data-protection rules. We do not claim certifications or a penetration test.

3.9 Threat summary

Threat	Controls
Account takeover	bcrypt, rate limits, 15-minute lockout, optional second factor, short-lived tokens, rotating refresh tokens
Cross-business data access	business_id scoping, Row-Level Security, object-level checks
Data breach	TLS, encryption at rest by the host, field-level encryption of contact details and bank tokens
Prompt injection	Data and instruction separation, read-only tools, figure checking
Unauthorized lender viewing	Explicit, scoped, expiring, revocable consent with an audit log
Injection attacks	ORM, input validation, API content security policy
Abuse of AI endpoints	Rate limits, step limits, request timeouts

4. Testing and deployment

- **Tests:** more than 260 automated tests (pytest) cover the engines, the API, the notification rules, the delivery layer, the credit planner, secret cleaning and PostgreSQL row-level security (opt-in, against a scratch database).
- **Migrations:** eight Alembic migrations create the schema; row-level security and the loan catalog are part of them.
- **Deployment:** Vercel (website), Render (API in a Docker container) and Neon (PostgreSQL), all on free tiers; step-by-step in `docs/deploy.md`. Optional services (email, WhatsApp, bank import, Stripe) switch on with environment variables and stay off otherwise.
- **Manual test cases:** `docs/manual-test-cases.md`.

5. Current limits and roadmap

Built and working	Not built yet (planned)
Forecast, what-if, Cash Advisor, collections and reminders, supplier plans, products, credit score and planner, loan center, reports, AI agent, alerts Staff logins, plans, two-factor, lender consent, row-level security CSV import, receipt scan, bank import (sandbox) Hosted deployment	Real lender integrations and a live loan marketplace Live payments for the Pro plan (billing is demo mode) Production bank connections and mobile-wallet import A verified-domain email sender and approved WhatsApp templates Malware scanning of uploads; unusual-login alerts; Redis rate limiter A learned credit model (gradient boosting with SHAP)